

stats-cli-py

AI Agent Skill Architecture Document

How We Built It | Best Practices | AI Agent Integration

Version: 1.2.1

Date: 2026-06-15

Commands: 33 | Tests: 794 | Coverage: 92%

Table of Contents

1. Skill Overview	What is stats-cli-py?
2. Architecture	Three-layer design
3. How AI Agents Call It	JSON dispatcher pattern
4. Progressive Disclosure	Two-layer design
5. Best Practices Score	8 criteria evaluation
6. Decision Trees	Statistical methodology routing
7. Self-Discovery	The discover command
8. Improvement History	What we improved

1. Skill Overview

stats-cli-py is a pure Python statistical analysis CLI/library designed for manufacturing and quality engineering. It provides 33 statistical commands covering descriptive statistics, hypothesis testing, SPC, capability analysis, DOE, MSA, reliability, multivariate analysis, time series, and more.

Key Design Principles:

- All I/O is JSON: stdin/stdout or file-based
- Pure Python: powered by scipy + statsmodels, no R dependency
- AI-Agent native: self-describing API with discover command
- Progressive disclosure: simple entry, deep on demand

Command Categories (33 commands)

Category	Commands
Basic	descriptive, normality, outlier
Hypothesis	ttest, anova, nonparametric, homogeneity, equivalence, power
SPC	control_chart, capability, trend
Regression	regression, correlation
DOE	doe (full_factorial, fractional, taguchi, RSM)
MSA	gage_rr (crossed, nested, attribute)
Reliability	reliability (weibull, kaplan_meier)
Multivariate	multivariate (pca, cluster, discriminant)
Time Series	timeseries (exp_smoothing, arima, decomposition)
Data	explore, clean, transform, discover
Workflow	workflow, workflow_template, check_assumptions, recommend
Report	report, export_excel, export_pdf, run

2. Architecture

The skill uses a three-layer architecture:

Layer 1: Router (main.py)

Parses JSON input, routes commands via `_route()` function with lazy imports, handles exceptions, wraps output in standard envelope. 374 lines.

Layer 2: Engine (stats_engine/*.py)

28 independent modules, each exposing one public function that accepts `**params` and returns a dict. All computation uses `scipy.stats`, `numpy`, and `statsmodels`. No custom statistical implementations.

Layer 3: Utils (utils/*.py)

4 utility modules: `data_loader` (file I/O with encoding detection), `data_cleaner` (NaN/Inf handling), `validators` (input validation), `output` (JSON envelope with rounding).

Data Flow

```
Input JSON -> main.py handler() -> _route(command, params)
    -> stats_engine/module.function(**params)
    -> utils/output.success(data) or error(msg)
    -> JSON stdout
```

3. How AI Agents Call It

Invocation Patterns

```
# Pattern 1: stdin pipe
echo '{"command":"descriptive","params":{"values":[1,2,3]}}' | python main.py

# Pattern 2: file argument
python main.py input.json

# Pattern 3: Python import
from main import handler
result = handler({"command": "descriptive", "params": {"values": [1,2,3]}})
```

Standard Output Envelope

```
Success: {"status":"success", "version":"1.2.1", "timestamp":"...", "data":{...}}
Error:   {"status":"error", "error_type":"...", "message":"...", "suggestion":"..."}
```

Agent Call Flow (7 Steps)

1. Agent receives vague user request ("analyze my data")
2. Agent calls discover -> gets 33 command summaries
3. Agent uses description/category to select command
4. Agent calls discover {command_name:"xxx"} -> gets param schema
5. Agent constructs JSON request using example as template
6. Agent invokes command -> gets result
7. Agent uses output_fields to parse and interpret result

4. Progressive Disclosure

The skill implements TWO layers of progressive disclosure:

Layer 1: SKILL.md Static Document (4 levels)

Level 1: Quick Start decision tree (5 top-level branches)

-> Level 2: 4 detailed decision trees (comparison, relationship, prediction, QC)

-> Level 3: 5 scenario-based workflows (complete JSON examples)

-> Level 4: 33 command reference (all parameters)

Layer 2: discover Runtime Dynamic Discovery (2 levels)

Level 1: discover (no params) -> lightweight summary (description + category)

-> Level 2: discover {command_name:"xxx"} -> full schema (params + output_fields)

Why Two Layers?

Static SKILL.md is loaded once when the skill is activated. It provides decision trees and scenarios for common cases. The discover command is queried at runtime for edge cases not covered by SKILL.md, or when the agent needs exact parameter schemas.

5. Best Practices Score

Criterion	Score	Key Strength
Trigger Conditions	Strong	Bilingual triggers, broad coverage
Progressive Disclosure	M-Strong	Two-layer design, decision trees
Decision Guidance	Strong	4 decision trees, recommend command
Error Handling	Moderate	7 error types, suggestion field
Example-Driven	Strong	5 scenarios, 33 command examples
Self-Documenting	Strong	Runtime discover with structured params
Composition	M-Strong	workflow, 9 templates, auto_check
Input Validation	Strong	Structured params with required/type

Key Design Wins

1. discover command: Runtime self-description, agents learn capabilities without reading docs
2. Decision trees: Encode correct statistical methodology (check normality first)
3. recommend command: Meta-guidance - agent can ask the tool what to use
4. workflow_template: 9 predefined templates for common analysis patterns
5. Structured params: required/type/desc for every parameter in discover

6. Decision Trees

Comparison Analysis Tree

```
User wants to compare data
|
+-- Only 2 groups?
|   +-- Normal? -> ttest (two_sample)
|   +-- Non-normal? -> nonparametric mann_whitney
|       +-- Also consider: transform boxcox -> retest
|
+-- 3+ groups?
|   +-- Normal? -> anova one_way
|   +-- Non-normal? -> nonparametric kruskal_wallis
|
+-- Paired data?
    +-- Normal? -> ttest paired
    +-- Non-normal? -> nonparametric wilcoxon
```

Quality Control Tree

```
User wants quality control
|
+-- Monitor stability? -> control_chart (imr/xbar/r)
+-- Check capability? -> capability (needs USL/LSL)
+-- Measurement system? -> gage_rr (crossed/nested)
+-- Reliability? -> reliability (weibull/kaplan_meier)
+-- Detect trends? -> trend (cusum/ewma/runs)
```


7. Self-Discovery (discover command)

The discover command is the runtime self-description mechanism. It allows AI agents to learn about available commands without reading SKILL.md.

Three Query Modes

```
# Mode 1: List all commands (lightweight)
{"command": "discover"}

# Mode 2: Filter by category
{"command": "discover", "params": {"category": "spc"}}

# Mode 3: Single command details (full schema)
{"command": "discover", "params": {"command_name": "capability"}}
```

Structured Parameters (v1.2.1)

Each command now exposes structured parameter metadata:

```
"params": [
  {"name": "values", "required": true, "type": "array", "desc": "Process data"},
  {"name": "usl", "required": true, "type": "number", "desc": "Upper spec limit"},
  {"name": "lsl", "required": true, "type": "number", "desc": "Lower spec limit"},
  {"name": "target", "required": false, "type": "number", "desc": "Target value"}
]
```

This eliminates guesswork - agents know exactly which parameters are required vs optional.

8. Improvement History

v1.2.1 Changes (2026-06-15)

SKILL.md Improvements:

- + Intent-to-Command quick lookup table (19 commands)
- + discover command recommendation at document top
- + Negative triggers (NOT for: streaming, text, image)
- + Top 5 command output examples
- + Error recovery workflow table
- + auto_check failure semantics documentation
- + run command sandbox restrictions
- + Box-Cox cross-links in decision trees
- + All placeholder [...] replaced with runnable data

Code Improvements:

- + Structured params in discover.py (33 commands)
- + fpdf2 deprecation fixes (report.py)
- + Matplotlib deprecation fix (charts.py)
- + Exception logging instead of swallowing

Packaging:

- + pyproject.toml: version 1.2.1, fpdf2 dependency, build-backend fix
- + Zip: excludes TODO.md, CLAUDE.md, tests, build artifacts
- + Final zip: 47 files, 310 KB